

Regresión Logística Multinomial

La regresión logística multinomial (o regresión *softmax*) es una generalización de la regresión logística, significa que la variable respuesta y puede tomar valores de entre múltiples clases:

$$y \in \{1, \dots, K\} \quad \text{donde } K \text{ es el \#clases}$$

en otras palabras, la regresión *softmax* es un modelo para problemas de clasificación multiclase ($K > 2$).

Definición del modelo

Al igual que la regresión logística, este modelo consta de tres componentes:

1. Componente aleatorio

Formado por **variables aleatorias categóricas** y_i , tal que:

$$y_i = k : k \in \{1, \dots, K\}$$

y un **vector de probabilidad** $\boldsymbol{\pi}_i = (\pi_{i1}, \dots, \pi_{iK})$ asociado a y_i , tal que:

$$\begin{aligned} \pi_{i1} + \dots + \pi_{iK} &= 1 \\ \sum_{k=1}^K \pi_{ik} &= 1 \quad i = 1, \dots, n \end{aligned}$$

- π_{ik} es la probabilidad, de que la i -ésima observación, pertenezca a la clase k .

En términos simples, cada observación $\mathbf{x}_i$ tiene asociado un vector de probabilidad $\boldsymbol{\pi}_i$, tal que la suma de sus elementos es 1.

El valor de y_i es la clase k con mayor probabilidad del vector $\boldsymbol{\pi}_i$,

2. Componente Sistemático

Consiste de *transformaciones lineales* (o conocido también como combinaciones lineales)

$$z_{ik} = \theta_{0k} + \theta_{1k}x_{i1} + \dots + \theta_{(p-1)k}x_{i(p-1)} \quad i = 1, \dots, n$$

expresado matricialmente, es:

$$z_{ik} = \boldsymbol{\theta}_k^T \mathbf{x}_i \quad i = 1, \dots, n; \quad k = 1, \dots, K$$

- $\boldsymbol{\theta}_k$ es el vector de parámetros ($1 \times p$) correspondiente a la clase k . Por tanto, habrá K vectores, uno por cada clase.
- $\mathbf{x}_i$ es el vector predictor correspondiente a la i -ésima observación, dicho vector consiste de p características, donde el primer elemento es 1.
- n es el número total de observaciones (de entrenamiento).
- K el número de clases.

3. Función enlace

Conecta el componente aleatorio y el componente sistémico a través de la función *Logit*.

Tomemos arbitrariamente una de las probabilidades como base, en este caso tomaremos la probabilidad de la primera clase π_{i1} , entonces:

$$\text{Logit}(\pi_{ik}) = \log\left(\frac{\pi_{ik}}{\pi_{i1}}\right) = z_{ik}$$

eliminando el logaritmo y despejando π_{ik} :

$$\pi_{ik} = \pi_{i1} e^{z_{ik}} \quad k = 2, \dots, K$$

tomando en cuenta la anterior expresión y considerando que $\pi_{i1} + \dots + \pi_{iK} = 1$, se puede llegar a las siguientes expresiones:

- Probabilidad de la clase 1:

$$\pi_{i1} = \frac{1}{1 + \sum_{j=2}^K e^{z_{ij}}}$$

- y para el resto de las clases:

$$\pi_{ik} = \frac{e^{z_{ik}}}{1 + \sum_{j=2}^K e^{z_{ij}}}, \quad k = 2, \dots, K$$

Nota: El desarrollo del cálculo de π_{i1} y π_{ik} se encuentra en Anexos.

Función Softmax

Realizando cálculos adicionales se puede unificar π_{i1} y π_{ik} , obteniendo así una única expresión que calcule la probabilidad para cualquier clase:

$$\pi_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^K e^{z_{ij}}}, \quad k = 1, \dots, K$$

esta expresión corresponde a la denominada **Función Softmax**.

Entrenamiento

El objetivo es encontrar los mejores parámetros θ_{kj} .

Estimador de máxima verosimilitud

Tomando el componente aleatorio podemos obtener una función de verosimilitud.

$$\begin{aligned} L(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_n) &= f(\boldsymbol{\pi}_1) \cdot \dots \cdot f(\boldsymbol{\pi}_n) \\ &= \prod_{i=1}^n f(\boldsymbol{\pi}_i) \end{aligned}$$

como $\boldsymbol{\pi}_i$ es un vector, entonces:

$$\begin{aligned} &= \prod_{i=1}^n f(\pi_{i1}, \pi_{i2}, \dots, \pi_{iK}) \\ &= \prod_{i=1}^n f(\pi_{i1}) \cdot f(\pi_{i2}) \cdot f(\pi_{iK}) \\ &= \prod_{i=1}^n \prod_{k=1}^K f(\pi_{ik}) \end{aligned}$$

finalmente:

$$L(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_n) = \prod_{i=1}^n \prod_{k=1}^K \pi_{ik}^{y_{ik}}$$

Nota: $f(\pi_{ik}) = \pi_{ik}^{y_{ik}}$ es una función indicatriz, así que $y_{ik} \in \{0, 1\}$.

Notar que cada y_i esta en codificación one-hot, por esta razón los y_{ik} solo tiene valores 0 o 1.

aplicando logaritmo a $L(\pi_{11}, \dots, \pi_{nK})$:

$$\log(L) = \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\pi_{ik})$$

escribiendo $\log(L) = l$ tenemos:

$$l(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_n) = \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\pi_{ik})$$

recordar que π_{ik} es una *función softmax* que depende de z_{ik} , además $z_{ik} = \boldsymbol{\theta}_k^T \mathbf{x}_i$, entonces la función $l(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_n)$ en realidad es una función que depende de los parámetros $\Theta = (\boldsymbol{\theta}_1^T, \dots, \boldsymbol{\theta}_K^T)$, por lo tanto podemos reescribir:

$$l(\boldsymbol{\pi}_1, \dots, \boldsymbol{\pi}_n) = l(\Theta)$$

De esta forma obtenemos un estimador de máxima verosimilitud para la matriz de parámetros Θ :

$$\hat{\Theta} = \underset{\theta}{\operatorname{argmax}} l(\Theta)$$

para definirlo en **términos de minimización**, reescribimos $J(\Theta) = -l(\Theta)$, obteniendo así una **función de costo**:

$$J(\Theta) = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\pi_{ik})$$

- recordar que π_{ik} es la *función softmax*.

ahora el objetivo es encontrar los parámetros θ_{jk} que **minimicen la función de costo**:

$$\hat{\Theta} = \underset{\theta}{\operatorname{argmin}} J(\Theta)$$

Estimación por el método Gradiente Descendente

Mediante este método podemos minimizar la función de costo y encontrar los mejores parámetros Θ .

1. Definir una *tasa de aprendizaje* α y un número de *epochs*.
2. Inicializar Θ como una **matriz** $(p \times K)$ con valores aleatorios.
3. Iterar el número de *epochs* y actualizar los valores de Θ en cada iteración:

```
for epoc to epochs do
     $\Theta := \Theta - \alpha \nabla J(\Theta)$ 
end
```

Luego, el reto es calcular el gradiente de la función de costo $\nabla J(\Theta)$.

Gradiente de la función de costo

$$\nabla J(\theta) = \frac{\partial J(\theta)}{\partial \theta}$$

omitiendo el desarrollo del cálculo del gradiente, el resultado expresando en forma matricial, es:

$$\nabla J(\theta) = -X^T(Y - \Pi)$$

- Π es una matriz $n \times K$ el cual contiene las probabilidades π_{ik} , (detalles de cómo calcular esta matriz, ver Anexos).
- Y es una matriz de $n \times K$, aplicar codificación *one-hot* (**importante**).
- X es una matriz de $n \times p$, debe contener 1's en la primera columna.
- Realizar una *normalización z-score* de los datos, excepto la columna de 1's

Predicción

Dado un nuevo conjunto de datos X estimar las respuestas $\hat{y}$.

El proceso de clasificación consiste en calcular las K probabilidades para cada $\mathbf{x}_i$:

$$\pi_{ik} = \frac{e^{\theta_k^T \mathbf{x}_i}}{\sum_{j=1}^K e^{\theta_j^T \mathbf{x}_i}}, \quad k = 1, \dots, K$$

luego la clase con mayor probabilidad, será la clase elegida como respuesta de la i -ésima observación ($\mathbf{x}_i$):

$$\hat{y}_i = \underset{k \in \{1, \dots, K\}}{\operatorname{argmax}} \pi_{ik}$$

En algunos casos será necesario definir un umbral para la probabilidad y así mejorar las predicciones.

Anexos

Cálculo de π_{i1} y π_{ik}

Tomemos en cuenta las siguientes expresiones:

$$\pi_{ik} = \pi_{i1} e^{z_{ik}} \quad k = 2, \dots, K \quad (1)$$

$$\pi_{i1} + \pi_{i2} + \dots + \pi_{iK} = 1 \quad (2)$$

reemplazando (1) en (2):

$$\begin{aligned}
 \pi_{i1} + \pi_{i1}e^{z_{i2}} + \dots + \pi_{i1}e^{z_{iK}} &= 1 \\
 \pi_{i1}(1 + e^{z_{i2}} + \dots + e^{z_{iK}}) &= 1 \\
 \pi_{i1}(1 + \sum_{j=2}^K e^{z_{ij}}) &= 1 \\
 \pi_{i1} &= \frac{1}{1 + \sum_{j=2}^K e^{z_{ij}}} \quad (3)
 \end{aligned}$$

reemplazando (3) en (1):

$$\begin{aligned}
 \pi_{ik} &= \frac{1}{1 + \sum_{j=2}^K e^{z_{ij}}} e^{z_{ik}} \\
 \pi_{ik} &= \frac{e^{z_{ik}}}{1 + \sum_{j=2}^K e^{z_{ij}}} \quad k = 2, \dots, K
 \end{aligned}$$

obteniendo así las probabilidades π_{i1} y π_{ik} con $i = 1, \dots, n$.

Cálculo de la matriz de probabilidades Π

Si bien se puede calcular cada probabilidad con la función softmax definida arriba, también podríamos **calcularlo en forma matricial**:

$$\text{softmax}(\mathbf{x}_i, \Theta) = \frac{e^{\mathbf{x}_i \Theta}}{\sum_{j=1}^K e^{\mathbf{x}_i \theta_j}}$$

- $\mathbf{x}_i$ es un vector $1 \times p$ correspondiente la observación i .
- θ_j es un vector $1 \times p$ correspondiente a los parámetros de una clases en específico.
- Θ es una matriz $p \times K$ correspondiente a todos los parámetros de todas las clases.

El resultado de esta función es un vector $1 \times K$.

El paquete *numpy* de Python permite hacer operaciones aritméticas entre matrices a través de un mecanismo llamado *Broadcasting*, por lo tanto, es posible **calcular directamente la matriz de probabilidades Π** definiendo una función softmax como:

$$\text{softmax}(X, \Theta) = \frac{e^{X\Theta}}{(\sum_{j=1}^K e^{\mathbf{x}_1 \theta_j}, \dots, \sum_{j=1}^K e^{\mathbf{x}_n \theta_j})}$$

- X es una matriz $n \times p$, es decir, correspondiente a las n observaciones y p variables predictoras. Recordar que la primera columna debe contener 1's.

- Θ es una matriz de parámetros $p \times K$.
- $\mathbf{x}_i$ es un vector $1 \times p$, que corresponde a la observación i .
- El denominador es un vector $n \times 1$.

El resultado es una matriz $n \times K$.

Referencias

- Softmax Regression <http://ufldl.stanford.edu/tutorial/supervised/SoftmaxRegression/>

By Carlos Ramos (2026)